

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Maxim Ormanji FAF-242

Report

Laboratory work No.7-8

Operating System

Checked by:

Reițman Petricia, *university assistant*

DISA, FCIM, UTM

Chișinău – 2026

Laboratory Work no. 7
Three Small Programs: Processes, Redirection, a Pipe

Purpose of the work

The purpose of this work is to run the answers to three questions from the lecture: how a process is born, where `printf` writes, and how `ls | wc -l` works. I compile and run three short C programs that do by hand what the shell does: `fork` + `exec` + `wait`, redirection with `close` + `open`, and a pipe with `pipe` + `dup`. I change one line in some of them and look at what happens.

Before you start

I created the working folder `~/os-lab7` and checked that the compiler is installed.

```
vboxuser@Ubuntu:~$ mkdir -p ~/os-lab7 && cd ~/os-lab7
vboxuser@Ubuntu:~/os-lab7$ gcc --version | head -1
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
```

Part 1. fork + exec + wait

The program `run.c` does what the shell does when I type `ls -l`: it copies itself (`fork`), the copy becomes `ls` (`exec`), and the original waits for it (`wait`).

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
/* the three steps of the shell for the command: ls -l */
int main(void){
    printf("parent: my PID is %d\n", getpid());
    fflush(stdout);
    pid_t pid = fork();                 /* step 1: copy myself */
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {                     /* only the child runs this */
        printf("child: my PID is %d, my parent is %d\n", getpid(), getppid());
        fflush(stdout);
        execlp("ls", "ls", "-l", (char *)NULL);   /* step 2: become ls */
        perror("exec");                 /* runs only if exec failed */
        exit(127);
    }
    int status;
    waitpid(pid, &status, 0);           /* step 3: wait for the child */
    printf("parent: child %d ended with status %d\n", pid, WEXITSTATUS(status));
    return 0;
}
```

I compiled and ran it:

```
vboxuser@Ubuntu:~/os-lab7$ gcc -Wall -Wextra -o run run.c
vboxuser@Ubuntu:~/os-lab7$ ./run
parent: my PID is 13820
child: my PID is 13821, my parent is 13820
total 24
-rwxrwxr-x 1 vboxuser vboxuser 16392 Nov  1 18:57 run
-rw-rw-r-- 1 vboxuser vboxuser   866 Nov  1 18:57 run.c
parent: child 13821 ended with status 0
```

Then I made the child run another program. I replaced the `execlp` line with `execlp("date", "date", (char *)NULL);` in a copy of the file, `run_date.c`, so that `run.c` stays with `ls`:

```
vboxuser@Ubuntu:~/os-lab7$ sed 's/execlp("ls", "ls", "-l"/execlp("date", "date"/' run.c >
run_date.c
vboxuser@Ubuntu:~/os-lab7$ gcc -Wall -Wextra -o run_date run_date.c
vboxuser@Ubuntu:~/os-lab7$ ./run_date
parent: my PID is 13828
child: my PID is 13829, my parent is 13828
Sun Nov  1 06:57:49 PM UTC 2026
parent: child 13829 ended with status 0
```

Then I asked for a program that does not exist:

```
vboxuser@Ubuntu:~/os-lab7$ sed 's/execlp("ls", "ls", "-l"/execlp("nosuchprog", "nosuchprog"/'
run.c > run_nosuch.c
vboxuser@Ubuntu:~/os-lab7$ gcc -Wall -Wextra -o run_nosuch run_nosuch.c
vboxuser@Ubuntu:~/os-lab7$ ./run_nosuch
parent: my PID is 13836
child: my PID is 13837, my parent is 13836
exec: No such file or directory
parent: child 13837 ended with status 127
```

Observe (Part 1)

1. **The line `perror("exec")` is in the child, right after `execlp`. Why is it printed only with `nosuchprog`?**
   When `exec` works, the code of the child process is replaced by the new program (`ls` or `date`), so the old code, including the line with `perror`, does not exist any more and `execlp` never returns. When the program does not exist, `exec` fails and returns to the old code, so the next line runs: `perror("exec")` prints `No such file or directory`, and `exit(127)` ends the child.
2. **Which number does the child print as "my parent", and whose PID is it?**
   In the last run the child (13837) prints `my parent is 13836`. It is the PID of the parent process, that is, of my `run_nosuch` program that called `fork` (the parent printed `my PID is 13836` in the first line). The same is true in the other runs (13820 → 13821, 13828 → 13829).
3. **What status does the parent see with `ls`, and what status with `nosuchprog`?**
   With `ls` the status is 0 (success). With `nosuchprog` it is 127: this is the number from `exit(127)` in the child after the failed `exec`, and it is the same code that the shell returned

for `command not found` in Lab 3.

Part 2. A zombie, on purpose

A zombie is a child that has ended but has not been waited for. I made one by adding `sleep(20);` before `waitpid`, so the parent sleeps and the child `ls` ends in the meantime. I made `zombie.c` from `run.c` (the version with `ls`) and checked that the new line is just before `waitpid`:

```
vboxuser@Ubuntu:~/os-lab7$ sed 's/^\( *\)waitpid(pid, &status, 0);/\1sleep(20);\n\1waitpid(pid
, \&status, 0);/' run.c > zombie.c
vboxuser@Ubuntu:~/os-lab7$ grep -n -B1 -A1 "sleep(20)" zombie.c
18-    int status;
19:    sleep(20);
20-    waitpid(pid, &status, 0);           /* step 3: wait for the child */
vboxuser@Ubuntu:~/os-lab7$ gcc -Wall -Wextra -o zombie zombie.c
```

I started it in the background and looked at the process table while the parent sleeps:

```
vboxuser@Ubuntu:~/os-lab7$ ./zombie > /dev/null &
[1] 13848
vboxuser@Ubuntu:~/os-lab7$ sleep 2; ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
  13455   13426 Ss   /usr/bin/bash
  13848   13455 S    ./zombie
  13850   13848 Z    [ls] <defunct>
  13855   13455 R+   ps -o pid,ppid,stat,cmd
```

After 20 seconds I ran `ps` again:

```
vboxuser@Ubuntu:~/os-lab7$ sleep 20; ps -o pid,ppid,stat,cmd
[1]+  Done                    ./zombie > /dev/null
    PID    PPID STAT CMD
  13455   13426 Ss   /usr/bin/bash
  13857   13455 R+   ps -o pid,ppid,stat,cmd
```

Observe (Part 2)

1. **Which letter does `ls` have in the STAT column, and what does it mean?**
   It has `Z` (the line is `[ls] <defunct>`). It means zombie: the process `ls` (PID 13850) has already ended, but its parent `./zombie` (PID 13848) has not called `wait` yet, so the system keeps its entry in the process table (with the exit status) until the parent collects it. The parent is in the state `S`, because it is sleeping in `sleep(20)`.
2. **After 20 seconds, is the `[ls] <defunct>` line still there? Which line of `zombie.c` made it disappear?**
   No, it is not there any more (the second `ps`), and `./zombie` also finished (`Done`). The line `waitpid(pid, &status, 0);` made it disappear: after `sleep(20)` the parent called `waitpid`, which took the exit status of the child and let the system remove its entry. After that the parent printed the message and ended.

Part 3. Redirection: ls -l > out.txt

The program `redirect.c` does what the shell does for `ls -l > out.txt`: in the child it closes descriptor 1, opens the file (which gets the smallest free number, 1), and then runs `ls`.

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>
/* does the same as the shell command: ls -l > out.txt */
int main(void)
{
    pid_t pid = fork();
    if (pid == 0) {
        close(1);                       /* descriptor 1 is now free */
        int fd = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fd < 0) { perror("open"); exit(1); }
        fprintf(stderr, "open gave descriptor %d\n", fd);
        execlp("ls", "ls", "-l", (char *)NULL);   /* ls writes to 1, as always */
        perror("exec");
        exit(127);
    }
    waitpid(pid, NULL, 0);
    printf("done: the listing is in out.txt\n");
    return 0;
}
```

I compiled it, ran it and looked inside the file:

```
vboxuser@Ubuntu:~/os-lab7$ gcc -Wall -Wextra -o redirect redirect.c
vboxuser@Ubuntu:~/os-lab7$ ./redirect
open gave descriptor 1
done: the listing is in out.txt
vboxuser@Ubuntu:~/os-lab7$ head -3 out.txt
total 116
-rw-r--r-- 1 vboxuser vboxuser     0 Nov  1 18:58 out.txt
-rwxrwxr-x 1 vboxuser vboxuser 16336 Nov  1 18:58 redirect
```

The file `out.txt` is shown with size 0 in its own listing: `ls` reads the directory when the file has just been created and is still empty.

Observe (Part 3)

1. **Which number did `open` give, and why exactly that one?**
   It gave descriptor 1 (`open gave descriptor 1`). The function `open` always returns the smallest free descriptor number. Descriptors 0 (stdin) and 2 (stderr) were still used, and the number 1 was just freed by `close(1)`, so it was the smallest free one.
2. **`ls` was not changed at all. Why does its listing not appear on the screen?**
   `ls` always writes its listing to descriptor 1 and does not know where it points. In the child, descriptor

1 now points to `out.txt` and not to the terminal, and `exec` keeps the open descriptors, so the listing went into the file. The message `done: ...` was printed by the parent, whose descriptor 1 was not changed and still points to the screen. The line `open gave descriptor 1` was written to stderr (descriptor 2), which still pointed to the screen too.

3. **To redirect the errors instead (like `2> err.txt`), which descriptor would you close before `open`?**
   Descriptor 2 (stderr): `close(2)`, then `open` gives the number 2 and everything that `ls` writes as an error goes to the file.

Part 4. A pipe: ls /etc | wc -l

The program `pipeline.c` builds the pipeline by hand: one pipe and two forks. In `ls`, descriptor 1 becomes the writing end `p[1]`, in `wc` descriptor 0 becomes the reading end `p[0]`. The function `dup` copies a descriptor into the smallest free number, exactly like `open`.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
/* does the same as the shell command: ls /etc | wc -l */
int main(void)
{
    int p[2];
    if (pipe(p) < 0) { perror("pipe"); return 1; }
    if (fork() == 0) {                  /* ls: its 1 becomes p[1] */
        close(1);
        dup(p[1]);                      /* smallest free number: 1 */
        close(p[0]);
        close(p[1]);
        execlp("ls", "ls", "/etc", (char *)NULL);
        perror("exec ls");
        exit(127);
    }
    if (fork() == 0) {                  /* wc: its 0 becomes p[0] */
        close(0);
        dup(p[0]);                      /* smallest free number: 0 */
        close(p[0]);
        close(p[1]);
        execlp("wc", "wc", "-l", (char *)NULL);
        perror("exec wc");
        exit(127);
    }
    close(p[0]);                        /* the parent uses neither end */
    close(p[1]);
    wait(NULL);
    wait(NULL);
    return 0;
}
```

I compiled it, ran it and compared with the real shell:

```
vboxuser@Ubuntu:~/os-lab7$ gcc -Wall -Wextra -o pipeline pipeline.c
vboxuser@Ubuntu:~/os-lab7$ ./pipeline
237
vboxuser@Ubuntu:~/os-lab7$ ls /etc | wc -l
237
```

Then I broke it on purpose. In the copy `pipeline_broken.c` I turned the line `close(p[1]);` in the parent (after the comment "the parent uses neither end") into a comment. I started the program in the background and looked at the process table:

```
vboxuser@Ubuntu:~/os-lab7$ sed '/the parent uses neither end/{n;s|close(p\[1\]);|/* close(p
[1]); */|}' pipeline.c > pipeline_broken.c
vboxuser@Ubuntu:~/os-lab7$ grep -n -A1 "parent uses neither" pipeline_broken.c
28:    close(p[0]);                        /* the parent uses neither end */
29-    /* close(p[1]); */
vboxuser@Ubuntu:~/os-lab7$ gcc -Wall -Wextra -o pipeline_broken pipeline_broken.c
vboxuser@Ubuntu:~/os-lab7$ ./pipeline_broken &
[1] 13911
vboxuser@Ubuntu:~/os-lab7$ ps -o pid,ppid,stat,cmd
    PID    PPID STAT CMD
  13455   13426 Ss   /usr/bin/bash
  13911   13455 S    ./pipeline_broken
  13914   13911 S    wc -l
  13921   13455 R+   ps -o pid,ppid,stat,cmd
vboxuser@Ubuntu:~/os-lab7$ kill %1
vboxuser@Ubuntu:~/os-lab7$ 237
[1]+  Terminated              ./pipeline_broken
```

The program does not end: `ls` is already gone, and `wc -l` is still there in the state `S`. Instead of waiting and pressing Ctrl-C, I stopped the parent with `kill`. After that its copy of `p[1]` was closed, `wc` got the end of the data and printed `237`, so no hanging process was left.

Observe (Part 4)

1. **Did `./pipeline` and `ls /etc | wc -l` give the same number?**
   Yes, both printed 237.
2. **After the change, which of the two children never ends, `ls` or `wc`?**
   `wc` never ends. In the process table the process `wc -l` (PID 13914) is still there, and `ls` has already finished and is not there.
3. **Why, in one sentence: what is `wc` waiting for?**
   `wc` waits for the end of the data (EOF) on the pipe, and EOF comes only when all the writing ends `p[1]` are closed, but the parent still keeps its own copy of `p[1]` open, so `wc` waits forever (and the parent waits for `wc` in the second `wait`).

Conclusion (Lab 7)

In this work I saw how the shell does its basic jobs with only a few system calls. With `fork`, `execlp` and `waitpid` I made a child that becomes `ls`, `date` or a program that does not exist, and saw that the parent gets the status 0 or 127 and that the code after a successful `exec` never runs. By delaying `waitpid` I made a zombie and saw it with the letter `Z` in `ps`, and then saw it disappear when the parent waited for it. With `close` and `open` I redirected the output of `ls` to a file without changing `ls`, because `open` returns the smallest free descriptor. With `pipe` and `dup` I built `ls /etc | wc -l` by hand and got the same number as the shell (237). When I did not close the writing end of the pipe in the parent, `wc` never got EOF and the pipeline hung, so I understood why every process must close the pipe ends that it does not use.

Laboratory Work no. 8
The Same Calls, Inside Your Own OS

Purpose of the work

The purpose of this work is to run the same program that I ran on Linux in Lab 7 (`fork` + `exec` + `wait`) inside xv6, the small OS that I build this semester, and to see that the system calls are the same. Then I make two processes talk to each other through two pipes (`pingpong`).

Before you start

I used my tree `~/xv6-riscv`. Every new program is a file `user/NAME.c` plus one line `$U/_NAME\` in the `UPROGS` list of the `Makefile`. The line starts with a TAB and nothing follows the backslash. I checked the new lines with `grep -P`:

```
vboxuser@Ubuntu:~$ cd ~/xv6-riscv
vboxuser@Ubuntu:~/xv6-riscv$ grep -nP '^\t\$U/_(pingpong|run|runls|runnosuch)\\$' Makefile
133:        $U/_pingpong\
134:        $U/_run\
135:        $U/_runls\
136:        $U/_runnosuch\
```

The check finds all four lines, so every line starts with a TAB and has nothing after the backslash.

Part 1. run, inside xv6

The same three steps as `run.c` in Lab 7, written for xv6: there is no glibc here, only the system calls of the kernel that I compiled. The child becomes `echo`. This is `user/run.c`:

```
#include "kernel/types.h"
#include "user/user.h"
// the three steps of the shell, inside xv6: echo hello from the child
int
main(int argc, char *argv[]){
  printf("parent: my pid is %d\n", getpid());
  int pid = fork();                     // step 1: copy myself
  if(pid < 0){
    printf("fork failed\n");
    exit(1);
  }
  if(pid == 0){                         // only the child runs this
    printf("child: my pid is %d\n", getpid());
    char *args[] = { "echo", "hello", "from", "the", "child", 0 };
    exec("echo", args);                 // step 2: become echo
    printf("exec failed\n");            // runs only if exec failed
    exit(1);
  }
  int status;
```

```
  wait(&status);                          // step 3: wait for the child
  printf("parent: child %d ended with status %d\n", pid, status);
  exit(0);
}
```

I started xv6 and ran it:

```
vboxuser@Ubuntu:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -
global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device
virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
$
```

To make the child run `ls`, I made a copy of the program, `user/runls.c`, where the two lines are changed to `char *args[] = { "ls", 0 };` and `exec("ls", args);`, so `run.c` stays with `echo`. The line `$U/_runls\` is in `UPROGS`.

```
vboxuser@Ubuntu:~/xv6-riscv$ sed -e 's/char \*args\[\] = {.*};/char *args[] = { "ls", 0 };/' -
e 's/exec("echo", args);/exec("ls", args);/' user/run.c > user/runls.c
vboxuser@Ubuntu:~/xv6-riscv$ sed -i 's|// step 2: become echo|// step 2: become ls|' user/
runls.c
vboxuser@Ubuntu:~/xv6-riscv$ grep -n 'args\[\]\|exec(' user/runls.c
14:    char *args[] = { "ls", 0 };
15:    exec("ls", args);                // step 2: become ls
```

After `make qemu`:

```
$ runls
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
run            2 3 35888
runls          2 4 35888
runnosuch      2 5 35896
args           2 6 35272
cat            2 7 36728
echo           2 8 35592
sleep          2 9 35280
forktest       2 10 18136
grep           2 11 44184
init           2 12 35976
kill           2 13 35512
```

```
ln              2 14 35304
ls              2 15 42880
mkdir           2 16 35568
rm              2 17 35552
sh              2 18 58312
stressfs        2 19 36416
usertests       2 20 206800
grind           2 21 51824
wc              2 22 37728
zombie          2 23 34848
mystat          2 24 36232
find            2 25 37880
logstress       2 26 37560
forphan         2 27 36352
dorphan         2 28 35800
sync            2 29 34944
console         3 30 0
parent: child 4 ended with status 0
$
```

The third copy, `user/runnosuch.c`, has the line `exec("nosuchprog", args);`. The line `$U/_runnosuch\` is in `UPROGS`.

```
vboxuser@Ubuntu:~/xv6-riscv$ sed 's/exec("echo", args);/exec("nosuchprog", args);/' user/run.c
 > user/runnosuch.c
vboxuser@Ubuntu:~/xv6-riscv$ sed -i 's|// step 2: become echo|// step 2: become nosuchprog|'
user/runnosuch.c
vboxuser@Ubuntu:~/xv6-riscv$ grep -n 'exec(' user/runnosuch.c
15:    exec("nosuchprog", args);                // step 2: become nosuchprog
```

After `make qemu`:

```
$ runnosuch
parent: my pid is 7
child: my pid is 8
exec failed
parent: child 8 ended with status 1
$
```

Observe (Part 1)

1. **Write the three calls that `run.c` uses in xv6 and the three calls that `run.c` used on Linux in Lab 7. What is the same?**
   In xv6 the three calls are `fork()`, `exec("echo", args)` and `wait(&status)`. On Linux (Lab 7) they were `fork()`, `execlp("ls", "ls", "-l", NULL)` and `waitpid(pid, &status, 0)`. The idea is the same: the three steps of the shell (copy myself, become another program, wait for the child), and in both systems the parent gets the PID of the child from `fork` and the child gets 0. Only the names and the arguments are a little different: the Linux `execlp` looks for the program in `PATH` by itself, and `waitpid` can wait for one chosen child, but the system calls under them do the same work. On Linux they are wrappers in glibc, in xv6 they are the calls of the kernel that I compiled.

2. **Who prints "hello from the child": the code of `run`, or `echo`?**
   `echo` prints it. The code of `run` prints only the lines with the PIDs. After a successful `exec` the child is no longer `run`: its memory is replaced by the program `echo`, which receives the arguments `hello from the child` and prints them. The line `exec failed` is not printed in the first run, because after a successful `exec` the old code is gone.
3. **Why is the status 1 here and 127 on Linux?**
   Because of the number in the `exit` line after `exec`. In the xv6 program it is `exit(1)`, and in the Linux program of Lab 7 it was `exit(127)`. The parent just prints the number that the child gave to `exit`: 1 in the `runnosuch` run and 127 in Lab 7.

Part 2. pingpong

The parent sends one byte to the child through the first pipe. The child prints `<pid>: received ping`, writes the byte back through the second pipe and exits. The parent reads it, prints `<pid>: received pong` and waits for the child. I completed the TODO lines with `read(p2c[0], &b, 1)`, `write(c2p[1], &b, 1)`, `write(p2c[1], &b, 1)` and `read(c2p[0], &b, 1)`. I gave `b` the start value `'x'`, so the parent does not send an uninitialized byte. This is `user/pingpong.c`:

```
#include "kernel/types.h"
#include "user/user.h"
int
main(int argc, char *argv[])
{
  int p2c[2], c2p[2];                   // parent-to-child, child-to-parent
  char b = 'x';
  pipe(p2c);
  pipe(c2p);
  if(fork() == 0){
    close(p2c[1]); close(c2p[0]);       // the ends the child does not use
    read(p2c[0], &b, 1);
    printf("%d: received ping\n", getpid());
    write(c2p[1], &b, 1);
    exit(0);
  }
  close(p2c[0]); close(c2p[1]);         // the ends the parent does not use
  write(p2c[1], &b, 1);
  read(c2p[0], &b, 1);
  printf("%d: received pong\n", getpid());
  wait(0);
  exit(0);
}
```

The line `$U/_pingpong\` is in `UPROGS` (see above). I started xv6 and ran the program:

```
vboxuser@Ubuntu:~/xv6-riscv$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -
global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device
virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0
```

```
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ pingpong
4: received ping
3: received pong
$
```

The line `ping` is printed first, by the child (pid 4). The line `pong` is printed second, by the parent (pid 3).

Observe (Part 2)

1. **Why do you need TWO pipes, and not one for both directions?**
   A pipe works in one direction only: what is written into the writing end is read from the reading end. If I used one pipe for both directions, the parent would write the byte and then read from the same pipe, and it could take its own byte back before the child does. The child could also not know whose data it reads. With two pipes every direction has its own channel: `p2c` (parent to child) and `c2p` (child to parent), and each process closes the two ends that it does not use.
2. **Nothing in your code says "wait until the child has replied". Which call makes the parent wait?**
   The call `read(c2p[0], &b, 1)` in the parent. The pipe `c2p` is empty until the child writes the byte, so `read` blocks and the parent sleeps until the child sends the byte back. This is why `pong` always comes after `ping`. The last `wait(0)` makes the parent wait until the child process has ended.

Conclusion (Lab 8)

In this work I saw that the system calls that I used on Linux in Lab 7 are the same inside xv6: `fork`, `exec` and `wait` do the three steps of the shell, and the child becomes `echo`, `ls` or fails with the status that its own `exit` gives (1 here, 127 in Lab 7). I saw that the line `hello from the child` and the listing are printed by the new program after `exec`, not by the code of `run`. With `pipe`, `read`, `write` and `close` I made two processes talk through two one-way pipes, and saw that the blocking `read` in the parent is what makes it wait for the answer of the child, so `ping` always comes before `pong`.